

HOOKS_SYSTEM_USER_GUIDE

Hooks System User Guide

HelixCode Phase 2, Feature 4

Version: 1.0 **Last Updated:** November 7, 2025 **Package:** `dev.helix.code/internal/hooks`

Table of Contents

1. [Overview](#)
 2. [Core Concepts](#)
 3. [Getting Started](#)
 4. [Hook Types](#)
 5. [Priority System](#)
 6. [Execution Modes](#)
 7. [Event System](#)
 8. [Manager Operations](#)
 9. [Advanced Features](#)
 10. [Common Use Cases](#)
 11. [Best Practices](#)
 12. [API Reference](#)
 13. [Integration Patterns](#)
 14. [FAQ](#)
 15. [Troubleshooting](#)
-

Overview

The Hooks System provides a flexible, event-driven architecture for HelixCode that enables you to register callbacks (hooks) that execute in response to lifecycle events. This system supports synchronous and asynchronous execution, priority-based ordering, conditional execution, and comprehensive lifecycle management.

Key Features

- **13 Built-in Hook Types:** Covering task, LLM, edit, build, test, and error lifecycle events
- **Priority-Based Execution:** 5 priority levels controlling execution order

- **Synchronous & Asynchronous:** Flexible execution modes with concurrency control
- **Conditional Execution:** Execute hooks only when conditions are met
- **Thread-Safe:** All operations protected by RWMutex for concurrent access
- **Lifecycle Callbacks:** OnCreate, OnRemove, OnExecute event handlers
- **Rich Event System:** Events with typed data, metadata, and context
- **Execution Statistics:** Track success rates, durations, and status counts
- **Tag-Based Organization:** Categorize and filter hooks by tags
- **Timeout Support:** Per-hook timeout configuration
- **Context Cancellation:** Proper cancellation propagation

When to Use Hooks

- **Logging & Monitoring:** Track when tasks start, complete, or fail
 - **Metrics Collection:** Gather performance data during operations
 - **Error Handling:** Centralized error detection and response
 - **Validation:** Check preconditions before operations
 - **Cleanup:** Perform cleanup after operations
 - **Notifications:** Send alerts on important events
 - **Testing:** Inject test behavior at lifecycle points
 - **Auditing:** Track all operations for compliance
 - **Plugin System:** Allow extensions to react to events
 - **Workflow Automation:** Chain operations based on events
-

Core Concepts

1. Hooks

A **Hook** is a function that executes in response to an event. Each hook has:

- **ID:** Unique identifier (auto-generated)
- **Name:** Human-readable name
- **Type:** What event triggers this hook
- **Handler:** The function to execute
- **Priority:** Execution order (higher = earlier)
- **Async:** Execute asynchronously or synchronously
- **Timeout:** Maximum execution duration
- **Condition:** Optional condition to check before execution
- **Tags:** Labels for categorization
- **Metadata:** Custom key-value data
- **Enabled:** Whether the hook is active

2. Events

An **Event** represents something that happened in the system:

- **Type**: What kind of event (matches hook type)
- **Data**: Map of event-specific data
- **Context**: Go context for cancellation
- **Timestamp**: When the event occurred
- **Source**: Where the event came from
- **Metadata**: Additional string metadata

3. Executor

The **Executor** handles hook execution:

- Sorts hooks by priority
- Executes synchronously or asynchronously
- Manages concurrency with semaphores
- Tracks execution results
- Triggers callbacks
- Handles context cancellation

4. Manager

The **Manager** organizes and triggers hooks:

- Registers/unregisters hooks
- Organizes hooks by type
- Enables/disables hooks
- Triggers hooks for events
- Provides query methods (by type, tag, etc.)
- Thread-safe for concurrent access
- Maintains execution statistics

Getting Started

Basic Example

```
package main

import (
    "context"
    "fmt"
    "dev.helix.code/internal/hooks"
```

```

)

func main() {
    // Create a manager
    manager := hooks.NewManager()

    // Create a hook
    hook := hooks.NewHook(
        "logger",
        hooks.HookTypeBeforeTask,
        func(ctx context.Context, event *hooks.Event) error {
            fmt.Printf("Task starting: %v\n", event.Data)
            return nil
        },
    )

    // Register the hook
    manager.Register(hook)

    // Create an event
    event := hooks.NewEvent(hooks.HookTypeBeforeTask)
    event.SetData("task_id", "123")
    event.SetData("task_name", "Build Project")

    // Trigger hooks for the event
    results := manager.TriggerEvent(event)

    // Check results
    for _, result := range results {
        fmt.Printf("%s: %s\n", result.HookName, result.Status)
    }
}

```

Output

```

Task starting: map[task_id:123 task_name:Build Project]
logger: completed

```

Hook Types

The system provides 13 built-in hook types:

Task Lifecycle

```
HookTypeBeforeTask // Before any task execution
HookTypeAfterTask  // After task completion
```

Use Cases: - Log task start/end - Validate task parameters - Track task duration - Update task status

Example:

```
beforeTask := hooks.NewHook("task-logger",
    hooks.HookTypeBeforeTask,
    func(ctx context.Context, event *hooks.Event) error {
        taskID, _ := event.GetData("task_id")
        log.Printf("Starting task: %v", taskID)
        return nil
    },
)
```

LLM Lifecycle

```
HookTypeBeforeLLM // Before LLM API call
HookTypeAfterLLM  // After LLM response
```

Use Cases: - Log prompts and responses - Track token usage - Measure LLM latency - Filter or modify prompts - Cache responses

Example:

```
beforeLLM := hooks.NewHook("llm-metrics",
    hooks.HookTypeBeforeLLM,
    func(ctx context.Context, event *hooks.Event) error {
        prompt, _ := event.GetData("prompt")
        tokens, _ := event.GetData("max_tokens")
        metrics.RecordLLMCall(prompt, tokens)
        return nil
    },
)
```

Edit Lifecycle

```
HookTypeBeforeEdit // Before file edit
HookTypeAfterEdit  // After file edit
```

Use Cases: - Backup files before editing - Validate edit permissions - Track file changes - Trigger linters

Example:

```
beforeEdit := hooks.NewHook("backup", hooks.HookTypeBeforeEdit,
    func(ctx context.Context, event *hooks.Event) error {
        file, _ := event.GetData("file")
        return backup.CreateBackup(file.(string))
    },
)
```

Build Lifecycle

```
HookTypeBeforeBuild // Before build starts
HookTypeAfterBuild  // After build completes
```

Use Cases: - Clean build artifacts - Check dependencies - Generate code - Publish build results

Example:

```
beforeBuild := hooks.NewHook("clean", hooks.HookTypeBeforeBuild,
    func(ctx context.Context, event *hooks.Event) error {
        return os.RemoveAll("./build")
    },
)
```

Test Lifecycle

```
HookTypeBeforeTest // Before tests run
HookTypeAfterTest  // After tests complete
```

Use Cases: - Setup test databases - Seed test data - Clean test environment - Report test results

Example:

```
afterTest := hooks.NewHook("report", hooks.HookTypeAfterTest,
    func(ctx context.Context, event *hooks.Event) error {
        passed, _ := event.GetData("passed")
        failed, _ := event.GetData("failed")
        return reporter.SendTestResults(passed, failed)
    },
)
```

Status Events

```
HookTypeOnError    // When an error occurs
HookTypeOnSuccess  // When operation succeeds
```

Use Cases: - Send error notifications - Log failures - Track success rates - Trigger recovery

Example:

```
onError := hooks.NewHook("alert", hooks.HookTypeOnError,
    func(ctx context.Context, event *hooks.Event) error {
        err, _ := event.GetData("error")
        return notifier.SendAlert(err)
    },
)
```

Custom Events

```
HookTypeCustom      // User-defined events
```

Use Cases: - Application-specific events - Plugin system hooks - Custom workflows

Example:

```
custom := hooks.NewHook("deploy", hooks.HookTypeCustom,
    func(ctx context.Context, event *hooks.Event) error {
        // Handle custom deployment event
        return nil
    },
)
```

Priority System

Hooks execute in priority order (highest first). This allows you to control the sequence of execution when multiple hooks respond to the same event.

Priority Levels

```
PriorityLowest   = 1    // Run last (cleanup, logging)
PriorityLow      = 25   // Low importance
PriorityNormal   = 50   // Default priority
PriorityHigh     = 75   // Important operations
PriorityHighest  = 100  // Run first (validation, setup)
```

Valid Range

Priorities can be any value from `PriorityLowest` (1) to `PriorityHighest` (100).

Examples

Validation Before Processing

```
// Run first - validate parameters
validator := hooks.NewHookWithPriority(
    "validator",
    hooks.HookTypeBeforeTask,
    func(ctx context.Context, event *hooks.Event) error {
        taskID, ok := event.GetData("task_id")
        if !ok {
            return fmt.Errorf("task_id required")
        }
        return nil
    },
    hooks.PriorityHighest, // 100 - runs first
)

// Run second - log the task
logger := hooks.NewHookWithPriority(
    "logger",
    hooks.HookTypeBeforeTask,
    func(ctx context.Context, event *hooks.Event) error {
        log.Printf("Task starting")
        return nil
    },
    hooks.PriorityNormal, // 50 - runs second
)

// Run last - track metrics
metrics := hooks.NewHookWithPriority(
    "metrics",
    hooks.HookTypeBeforeTask,
    func(ctx context.Context, event *hooks.Event) error {
        metricsCollector.Increment("tasks_started")
        return nil
    },
    hooks.PriorityLowest, // 1 - runs last
)
```


Execution Order

When you trigger HookTypeBeforeTask: 1. **validator** (priority 100) - validates parameters 2. **logger** (priority 50) - logs the task 3. **metrics** (priority 1) - updates metrics

Custom Priorities

You can use any value in the valid range:

```
hook.Priority = 42 // Custom priority
hook.Priority = hooks.PriorityHigh + 5 // Slightly higher than high
```

Updating Priority

```
manager.UpdatePriority(hook.ID, hooks.PriorityHigh)
```

Execution Modes

Synchronous Execution

Hooks execute in the caller's goroutine and block until complete.

When to Use: - Hook must complete before continuing - Order matters - Simple, fast operations - Error handling is critical

Example:

```
syncHook := hooks.NewHook("validator", hooks.HookTypeBeforeTask,
    func(ctx context.Context, event *hooks.Event) error {
        // Validate before task runs
        return validateTask(event)
    },
)
// Async defaults to false
```

Asynchronous Execution

Hooks execute in separate goroutines and don't block the caller.

When to Use: - Long-running operations - I/O-bound operations - Independent side effects - Notifications - Metrics collection

Example:

```

asyncHook := hooks.NewAsyncHook("notifier",
    hooks.HookTypeAfterTask,
    func(ctx context.Context, event *hooks.Event) error {
        // Send notification asynchronously
        return sendNotification(event)
    },
)

```

Concurrency Control

The executor limits concurrent async executions:

```

executor := hooks.NewExecutorWithLimit(5) // Max 5 concurrent
manager := hooks.NewManagerWithExecutor(executor)

```

Default: 10 concurrent async executions

Waiting for Async Hooks

```

// Trigger and continue
results := manager.TriggerEvent(event)

// Later, wait for all async hooks to complete
manager.Wait()

```

Trigger and Wait

```

// Trigger and wait for ALL hooks (including async) to complete
results := manager.TriggerEventAndWait(event)

```

Synchronous Trigger

Force all hooks to execute synchronously:

```

// Even async hooks execute synchronously
results := manager.TriggerEventSync(event)

```

Event System

Creating Events

```

// Simple event
event := hooks.NewEvent(hooks.HookTypeBeforeTask)

```

```
// Event with context
ctx := context.Background()
event := hooks.NewEventWithContext(ctx, hooks.HookTypeBeforeTask)
```

Event Data

Store any type of data in events:

```
event.SetData("task_id", "123")
event.SetData("task_name", "Build Project")
event.SetData("config", ConfigStruct{...})
event.SetData("count", 42)
```

```
// Retrieve data
taskID, ok := event.GetData("task_id")
if ok {
    fmt.Printf("Task ID: %v\n", taskID)
}
```

Event Metadata

String key-value pairs for searchable data:

```
event.SetMetadata("user", "alice")
event.SetMetadata("environment", "production")
```

```
user, ok := event.GetMetadata("user")
```

Event Properties

```
event.Type           // HookType
event.Timestamp      // time.Time - when created
event.Source         // string - event source
event.Context        // context.Context
```

Setting Event Source

```
event.Source = "task-manager"
event.Source = "llm-service"
event.Source = "build-system"
```

Manager Operations

Creating a Manager

```
// Default manager
manager := hooks.NewManager()

// Manager with custom executor
executor := hooks.NewExecutorWithLimit(20)
manager := hooks.NewManagerWithExecutor(executor)
```

Registering Hooks

```
hook := hooks.NewHook("logger", hooks.HookTypeBeforeTask,
    handler)
err := manager.Register(hook)
if err != nil {
    // Handle error (duplicate ID, validation failed)
}
```

Register Many:

```
hooksToRegister := []*hooks.Hook{hook1, hook2, hook3}
err := manager.RegisterMany(hooksToRegister)
```

Unregistering Hooks

```
err := manager.Unregister(hook.ID)
if err != nil {
    // Hook not found
}
```

Getting Hooks

By ID:

```
hook, err := manager.Get("hook-id")
```

By Type:

```
beforeTaskHooks := manager.GetByType(hooks.HookTypeBeforeTask)
```

By Tag:

```
loggingHooks := manager.GetByTag("logging")
```

All Hooks:

```
allHooks := manager.GetAll()
```

Enabled Only:

```
enabledHooks := manager.GetEnabled()
```

By Name (substring match):

```
hooks := manager.FindByName("logger")
```

Enabling/Disabling Hooks

```
// Disable single hook  
manager.Disable(hook.ID)
```

```
// Enable single hook  
manager.Enable(hook.ID)
```

```
// Disable all hooks  
manager.DisableAll()
```

```
// Enable all hooks  
manager.EnableAll()
```

Counting Hooks

```
total := manager.Count()  
beforeTaskCount := manager.CountByType(hooks.HookTypeBeforeTask)
```

Clearing Hooks

```
manager.Clear() // Remove all hooks
```

Triggering Hooks

Basic Trigger:

```
ctx := context.Background()  
results := manager.Trigger(ctx, hooks.HookTypeBeforeTask)
```

With Event:

```
event := hooks.NewEvent(hooks.HookTypeBeforeTask)  
event.SetData("task_id", "123")  
results := manager.TriggerEvent(event)
```

Trigger and Wait:

```
// Wait for all hooks (including async)
results := manager.TriggerAndWait(ctx, hooks.HookTypeBeforeTask)
```

Synchronous Trigger:

```
// Force synchronous execution
results := manager.TriggerSync(ctx, hooks.HookTypeBeforeTask)
```

Statistics

```
stats := manager.GetStatistics()
fmt.Printf("Total: %d\n", stats.TotalHooks)
fmt.Printf("Enabled: %d\n", stats.EnabledHooks)
fmt.Printf("Disabled: %d\n", stats.DisabledHooks)
fmt.Printf("By Type: %v\n", stats.ByType)
fmt.Printf("Executor: %s\n", stats.ExecutorStats.String())
```

Advanced Features

Conditional Execution

Execute hooks only when conditions are met:

```
hook := hooks.NewHook("prod-only", hooks.HookTypeBeforeTask,
    handler)
hook.Condition = func(event *hooks.Event) bool {
    env, _ := event.GetMetadata("environment")
    return env == "production"
}
```

Common Patterns:

Environment Check:

```
hook.Condition = func(event *hooks.Event) bool {
    env, _ := event.GetMetadata("environment")
    return env == "production" || env == "staging"
}
```

Data Presence:

```
hook.Condition = func(event *hooks.Event) bool {
    _, ok := event.GetData("user_id")
}
```

```
    return ok // Only if user_id present
}
```

Tag-Based:

```
hook.Condition = func(event *hooks.Event) bool {
    critical, _ := event.GetData("critical")
    return critical == true
}
```

Timeouts

Set maximum execution time per hook:

```
hook := hooks.NewHook("slow-op", hooks.HookTypeBeforeBuild,
    handler)
hook.Timeout = 5 * time.Second

// Handler must respect context cancellation
handler := func(ctx context.Context, event *hooks.Event) error {
    select {
    case <-time.After(10 * time.Second):
        // Would timeout
        return nil
    case <-ctx.Done():
        // Properly handle timeout
        return ctx.Err()
    }
}
```

Important: Handlers must check `ctx.Done()` to respect timeouts.

Tags

Organize hooks with tags:

```
hook := hooks.NewHook("logger", hooks.HookTypeBeforeTask,
    handler)
hook.AddTag("logging")
hook.AddTag("critical")
hook.AddTag("production")

// Check tags
if hook.HasTag("logging") {
    // ...
}
```

```
// Get all hooks with tag
loggingHooks := manager.GetByTag("logging")
```

Metadata

Store custom metadata on hooks:

```
hook.SetMetadata("author", "alice")
hook.SetMetadata("version", "1.2.0")
hook.SetMetadata("ticket", "PROJ-123")
```

```
// Retrieve
author, ok := hook.GetMetadata("author")
```

Cloning Hooks

Create copies of hooks:

```
original := hooks.NewHook("logger", hooks.HookTypeBeforeTask,
    handler)
clone := original.Clone()
clone.Name = "logger-copy"
clone.Priority = hooks.PriorityHigh

manager.Register(clone)
```

Manager Clone:

```
clone, err := manager.Clone(hook.ID)
```

Lifecycle Callbacks

React to hook management events:

OnCreate:

```
manager.OnCreate(func(hook *hooks.Hook) {
    log.Printf("Hook registered: %s\n", hook.Name)
})
```

OnRemove:

```
manager.OnRemove(func(hook *hooks.Hook) {
    log.Printf("Hook unregistered: %s\n", hook.Name)
})
```


OnExecute:

```
manager.OnExecute(func(event *hooks.Event, results
    []*hooks.ExecutionResult) {
    log.Printf("Executed %d hooks for %s\n", len(results),
        event.Type)
})
```

Execution Results

Each hook execution returns a result:

```
type ExecutionResult struct {
    HookID      string      // Hook identifier
    HookName    string      // Hook name
    Status      HookStatus    // Status (pending, running,
        completed, failed, canceled, skipped)
    Error       error        // Error if failed
    Duration    time.Duration // How long it took
    StartedAt   time.Time     // When started
    CompletedAt time.Time     // When completed
}
```

Processing Results:

```
results := manager.TriggerEvent(event)
for _, result := range results {
    switch result.Status {
    case hooks.StatusCompleted:
        log.Printf("✓ %s completed in %v\n", result.HookName,
            result.Duration)
    case hooks.StatusFailed:
        log.Printf("✗ %s failed: %v\n", result.HookName,
            result.Error)
    case hooks.StatusSkipped:
        log.Printf("- %s skipped\n", result.HookName)
    }
}
```

Executor Statistics

Track execution metrics:

```
executor := manager.GetExecutor()
stats := executor.GetStatistics()
```

```
fmt.Printf("Total Executions: %d\n", stats.TotalExecutions)
fmt.Printf("Success Rate: %.1f%%\n", stats.SuccessRate * 100)
fmt.Printf("Average Duration: %v\n", stats.AverageDuration)
fmt.Printf("By Status: %v\n", stats.ByStatus)
```

Executor Callbacks

React to execution events:

OnComplete:

```
executor.OnComplete(func(result *hooks.ExecutionResult) {
    log.Printf("Completed: %s\n", result.HookName)
})
```

OnError:

```
executor.OnError(func(result *hooks.ExecutionResult) {
    log.Printf("Failed: %s - %v\n", result.HookName,
        result.Error)
    notifier.SendAlert(result.Error)
})
```

Export/Import

Export hook metadata (without handlers):

```
metadata := manager.Export()
// Returns []*HookMetadata

// Each metadata contains:
// ID, Name, Type, Description, Priority, Async, Enabled, Tags,
// Metadata
```

Common Use Cases

1. Logging System

Track all operations with hooks:

```
func setupLogging(manager *hooks.Manager) {
    // Log task starts
    manager.Register(hooks.NewHook("log-task-start",
        hooks.HookTypeBeforeTask,
        func(ctx context.Context, event *hooks.Event) error {
```

```

        taskID, _ := event.GetData("task_id")
        log.Printf("[TASK] Starting: %v\n", taskID)
        return nil
    },
))

// Log task completions
manager.Register(hooks.NewHook("log-task-end",
    hooks.HookTypeAfterTask,
    func(ctx context.Context, event *hooks.Event) error {
        taskID, _ := event.GetData("task_id")
        duration, _ := event.GetData("duration")
        log.Printf("[TASK] Completed: %v (took %v)\n",
            taskID, duration)
        return nil
    },
))

// Log errors
manager.Register(hooks.NewHook("log-errors",
    hooks.HookTypeOnError,
    func(ctx context.Context, event *hooks.Event) error {
        err, _ := event.GetData("error")
        source, _ := event.GetData("source")
        log.Printf("[ERROR] %s: %v\n", source, err)
        return nil
    },
))
}

```

2. Metrics Collection

Collect performance metrics:

```

type MetricsCollector struct {
    taskDurations []time.Duration
    llmCalls      int
    errors        int
}

func (m *MetricsCollector) Setup(manager *hooks.Manager) {
    // Track task durations
    manager.Register(hooks.NewAsyncHook("metrics-tasks",
        hooks.HookTypeAfterTask,
        func(ctx context.Context, event *hooks.Event) error {

```

```

        duration, _ := event.GetData("duration")
        m.taskDurations = append(m.taskDurations, duration.
(time.Duration))
        return nil
    },
))

// Count LLM calls
manager.Register(hooks.NewAsyncHook("metrics-llm",
    hooks.HookTypeBeforeLLM,
    func(ctx context.Context, event *hooks.Event) error {
        m.llmCalls++
        return nil
    },
))

// Count errors
manager.Register(hooks.NewAsyncHook("metrics-errors",
    hooks.HookTypeOnError,
    func(ctx context.Context, event *hooks.Event) error {
        m.errors++
        return nil
    },
))
}

func (m *MetricsCollector) Report() {
    fmt.Printf("Tasks: %d, LLM Calls: %d, Errors: %d\n",
        len(m.taskDurations), m.llmCalls, m.errors)
}

```

3. Notification System

Send alerts on important events:

```

func setupNotifications(manager *hooks.Manager, notifier
    *Notifier) {
    // Alert on errors
    manager.Register(hooks.NewAsyncHook("alert-errors",
        hooks.HookTypeOnError,
        func(ctx context.Context, event *hooks.Event) error {
            err, _ := event.GetData("error")
            return notifier.SendError(err)
        },
    ))
}

```

```

// Notify on build completion
manager.Register(hooks.NewAsyncHook("notify-build",
    hooks.HookTypeAfterBuild,
    func(ctx context.Context, event *hooks.Event) error {
        success, _ := event.GetData("success")
        if success.(bool) {
            return notifier.SendSuccess("Build completed")
        }
        return notifier.SendWarning("Build failed")
    },
))

// Daily reports
manager.Register(hooks.NewAsyncHook("daily-report",
    hooks.HookTypeCustom,
    func(ctx context.Context, event *hooks.Event) error {
        report, _ := event.GetData("report")
        return notifier.SendReport(report)
    },
))
}

```

4. Validation Pipeline

Validate operations before execution:

```

func setupValidation(manager *hooks.Manager) {
    // Validate task parameters (highest priority)
    validator := hooks.NewHookWithPriority(
        "validate-task",
        hooks.HookTypeBeforeTask,
        func(ctx context.Context, event *hooks.Event) error {
            taskId, ok := event.GetData("task_id")
            if !ok {
                return fmt.Errorf("task_id required")
            }

            taskType, ok := event.GetData("task_type")
            if !ok {
                return fmt.Errorf("task_type required")
            }

            validTypes := []string{"build", "test", "deploy"}
            if !contains(validTypes, taskType.(string)) {

```

```

        return fmt.Errorf("invalid task_type: %v",
taskType)
    }

    return nil
},
hooks.PriorityHighest,
)
manager.Register validator)

// Validate file edits
manager.Register(hooks.NewHookWithPriority(
    "validate-edit",
    hooks.HookTypeBeforeEdit,
    func(ctx context.Context, event *hooks.Event) error {
        file, _ := event.GetData("file")
        if !fileExists(file.(string)) {
            return fmt.Errorf("file not found: %s", file)
        }
        return nil
    },
    hooks.PriorityHighest,
))
}

```

5. Audit Trail

Track all operations for compliance:

```

type AuditLogger struct {
    db *Database
}

func (a *AuditLogger) Setup(manager *hooks.Manager) {
    // Audit all task executions
    manager.Register(hooks.NewAsyncHook("audit-task",
        hooks.HookTypeAfterTask,
        func(ctx context.Context, event *hooks.Event) error {
            user, _ := event.GetMetadata("user")
            taskID, _ := event.GetData("task_id")

            return a.db.LogAudit(AuditRecord{
                User:      user,
                Action:    "task_executed",
                Resource: taskID.(string),
            })
        }
    ))
}

```

```

        Timestamp: event.Timestamp,
    })
},
))

// Audit file edits
manager.Register(hooks.NewAsyncHook("audit-edit",
    hooks.HookTypeAfterEdit,
    func(ctx context.Context, event *hooks.Event) error {
        user, _ := event.GetMetadata("user")
        file, _ := event.GetData("file")

        return a.db.LogAudit(AuditRecord{
            User:      user,
            Action:    "file_edited",
            Resource: file.(string),
            Timestamp: event.Timestamp,
        })
    },
))
}

```

6. Caching System

Cache LLM responses:

```

type LLMCache struct {
    cache map[string]string
    mu     sync.RWMutex
}

func (c *LLMCache) Setup(manager *hooks.Manager) {
    // Check cache before LLM call
    manager.Register(hooks.NewHookWithPriority(
        "cache-check",
        hooks.HookTypeBeforeLLM,
        func(ctx context.Context, event *hooks.Event) error {
            prompt, _ := event.GetData("prompt")

            c.mu.RLock()
            cached, found := c.cache[prompt.(string)]
            c.mu.RUnlock()

            if found {
                event.SetData("cached_response", cached)
            }
        }
    ))
}

```

```

        event.SetData("cache_hit", true)
    }
    return nil
},
hooks.PriorityHighest,
))

// Store response in cache
manager.Register(hooks.NewAsyncHook("cache-store",
    hooks.HookTypeAfterLLM,
    func(ctx context.Context, event *hooks.Event) error {
        // Skip if cache hit
        if hit, _ := event.GetData("cache_hit"); hit == true
        {
            return nil
        }

        prompt, _ := event.GetData("prompt")
        response, _ := event.GetData("response")

        c.mu.Lock()
        c.cache[prompt.(string)] = response.(string)
        c.mu.Unlock()

        return nil
    },
))
}

```

7. Resource Cleanup

Clean up resources after operations:

```

func setupCleanup(manager *hooks.Manager) {
    // Clean build artifacts
    manager.Register(hooks.NewHook("cleanup-build",
        hooks.HookTypeAfterBuild,
        func(ctx context.Context, event *hooks.Event) error {
            // Delete temporary files
            os.RemoveAll("/tmp/build")
            return nil
        },
    ))

    // Clean test data
}

```



```

manager.Register(hooks.NewHook("cleanup-test",
    hooks.HookTypeAfterTest,
    func(ctx context.Context, event *hooks.Event) error {
        // Drop test database
        return dropTestDatabase()
    },
))
}

```

8. Error Recovery

Automatic error recovery:

```

func setupErrorRecovery(manager *hooks.Manager) {
    manager.Register(hooks.NewHook("recover",
        hooks.HookTypeOnError,
        func(ctx context.Context, event *hooks.Event) error {
            err, _ := event.GetData("error")
            taskID, _ := event.GetData("task_id")

            // Log error
            log.Printf("Task %v failed: %v\n", taskID, err)

            // Attempt recovery
            if isRetryable(err) {
                return retryTask(taskID.(string))
            }

            // Notify team
            return notifier.SendAlert(err)
        },
    ))
}

```

Best Practices

1. Hook Naming

Use descriptive names:

```

// Good
"validate-task-params"
"log-llm-call"
"notify-build-complete"

```

```
// Bad
"hook1"
"test"
"myHook"
```

2. Error Handling

Always handle errors in hooks:

```
// Good
func(ctx context.Context, event *hooks.Event) error {
    if err := doSomething(); err != nil {
        log.Printf("Hook failed: %v\n", err)
        return fmt.Errorf("failed to do something: %w", err)
    }
    return nil
}
```

```
// Bad
func(ctx context.Context, event *hooks.Event) error {
    doSomething() // Ignores errors
    return nil
}
```

3. Context Cancellation

Respect context cancellation in long-running hooks:

```
func(ctx context.Context, event *hooks.Event) error {
    for i := 0; i < 1000; i++ {
        select {
        case <-ctx.Done():
            return ctx.Err() // Respect cancellation
        default:
            processItem(i)
        }
    }
    return nil
}
```

4. Use Async for Independent Operations

If the hook doesn't need to block, make it async:

```
// Good - async notification
manager.Register(hooks.NewAsyncHook("notify",
    hooks.HookTypeAfterTask,
    func(ctx context.Context, event *hooks.Event) error {
        return sendNotification(event)
    },
))

// Bad - sync notification blocks task completion
manager.Register(hooks.NewHook("notify", hooks.HookTypeAfterTask,
    func(ctx context.Context, event *hooks.Event) error {
        return sendNotification(event) // Blocks
    },
))
```

5. Set Appropriate Priorities

Use priorities to control execution order:

```
// Validation first (highest)
validator.Priority = hooks.PriorityHighest

// Processing in middle (normal)
processor.Priority = hooks.PriorityNormal

// Logging last (lowest)
logger.Priority = hooks.PriorityLowest
```

6. Use Conditions for Targeted Execution

Don't execute hooks unnecessarily:

```
hook.Condition = func(event *hooks.Event) bool {
    env, _ := event.GetMetadata("environment")
    return env == "production" // Only in production
}
```

7. Tag for Organization

Use tags to group related hooks:

```
hook.AddTag("logging")
hook.AddTag("production")
hook.AddTag("critical")
```

```
// Later, disable all logging hooks
loggingHooks := manager.GetByTag("logging")
for _, h := range loggingHooks {
    manager.Disable(h.ID)
}
```

8. Set Timeouts for Slow Operations

Prevent hooks from hanging:

```
hook.Timeout = 5 * time.Second
```

```
// Handler must respect timeout
handler := func(ctx context.Context, event *hooks.Event) error {
    select {
    case <-doSlowOperation():
        return nil
    case <-ctx.Done():
        return ctx.Err()
    }
}
```

9. Monitor Execution Statistics

Track hook performance:

```
// Periodically check stats
stats := manager.GetStatistics()
if stats.ExecutorStats.SuccessRate < 0.9 {
    log.Printf("WARNING: Hook success rate below 90%%\n")
}
```

10. Clean Up Resources

Always clean up in hooks:

```
func(ctx context.Context, event *hooks.Event) error {
    file, err := os.Open("data.txt")
    if err != nil {
        return err
    }
    defer file.Close() // Always close

    // Process file
}
```

```
    return nil
}
```

API Reference

Hook Creation Functions

```
// Create basic hook
func NewHook(name string, hookType HookType, handler HookFunc)
    *Hook

// Create async hook
func NewAsyncHook(name string, hookType HookType, handler
    HookFunc) *Hook

// Create hook with priority
func NewHookWithPriority(name string, hookType HookType, handler
    HookFunc, priority HookPriority) *Hook
```

Hook Methods

```
func (h *Hook) AddTag(tag string)
func (h *Hook) HasTag(tag string) bool
func (h *Hook) SetMetadata(key, value string)
func (h *Hook) GetMetadata(key string) (string, bool)
func (h *Hook) ShouldExecute(event *Event) bool
func (h *Hook) Execute(ctx context.Context, event *Event) error
func (h *Hook) Clone() *Hook
func (h *Hook) String() string
func (h *Hook) Validate() error
```

Event Creation Functions

```
// Create basic event
func NewEvent(eventType HookType) *Event

// Create event with context
func NewEventWithContext(ctx context.Context, eventType
    HookType) *Event
```

Event Methods

```
func (e *Event) SetData(key string, value interface{})
func (e *Event) GetData(key string) (interface{}, bool)
```

```
func (e *Event) SetMetadata(key, value string)  
func (e *Event) GetMetadata(key string) (string, bool)  
func (e *Event) String() string
```

Manager Creation Functions

```
// Create default manager  
func NewManager() *Manager  
  
// Create manager with custom executor  
func NewManagerWithExecutor(executor *Executor) *Manager
```

Manager Registration Methods

```
func (m *Manager) Register(hook *Hook) error  
func (m *Manager) RegisterMany(hooks []*Hook) error  
func (m *Manager) Unregister(id string) error
```

Manager Query Methods

```
func (m *Manager) Get(id string) (*Hook, error)  
func (m *Manager) GetByType(hookType HookType) []*Hook  
func (m *Manager) GetByTag(tag string) []*Hook  
func (m *Manager) GetAll() []*Hook  
func (m *Manager) GetEnabled() []*Hook  
func (m *Manager) FindByName(nameSubstring string) []*Hook
```

Manager Control Methods

```
func (m *Manager) Enable(id string) error  
func (m *Manager) Disable(id string) error  
func (m *Manager) EnableAll()  
func (m *Manager) DisableAll()  
func (m *Manager) Clear()
```

Manager Trigger Methods

```
func (m *Manager) Trigger(ctx context.Context, eventType  
    HookType) []*ExecutionResult  
func (m *Manager) TriggerEvent(event *Event) []*ExecutionResult  
func (m *Manager) TriggerAndWait(ctx context.Context, eventType  
    HookType) []*ExecutionResult  
func (m *Manager) TriggerEventAndWait(event *Event)  
    []*ExecutionResult
```

```
func (m *Manager) TriggerSync(ctx context.Context, eventType
    HookType) []*ExecutionResult
func (m *Manager) TriggerEventSync(event *Event)
    []*ExecutionResult
func (m *Manager) Wait()
```

Manager Statistics Methods

```
func (m *Manager) Count() int
func (m *Manager) CountByType(hookType HookType) int
func (m *Manager) GetStatistics() *ManagerStatistics
func (m *Manager) Export() []*HookMetadata
```

Manager Modification Methods

```
func (m *Manager) UpdatePriority(id string, priority
    HookPriority) error
func (m *Manager) Clone(id string) (*Hook, error)
```

Manager Callback Methods

```
func (m *Manager) OnCreate(callback HookCallback)
func (m *Manager) OnRemove(callback HookCallback)
func (m *Manager) OnExecute(callback ExecuteCallback)
```

Executor Creation Functions

```
// Create default executor (10 concurrent)
func NewExecutor() *Executor

// Create executor with custom limit
func NewExecutorWithLimit(maxConcurrent int) *Executor
```

Executor Execution Methods

```
func (e *Executor) Execute(ctx context.Context, hook *Hook,
    event *Event) *ExecutionResult
func (e *Executor) ExecuteAll(ctx context.Context, hooks
    []*Hook, event *Event) []*ExecutionResult
func (e *Executor) ExecuteSync(ctx context.Context, hooks
    []*Hook, event *Event) []*ExecutionResult
func (e *Executor) ExecuteAndWait(ctx context.Context, hooks
    []*Hook, event *Event) []*ExecutionResult
func (e *Executor) Wait()
```

```
func (e *Executor) ExecuteWithTimeout(hook *Hook, event *Event,
    timeout time.Duration) *ExecutionResult
func (e *Executor) ExecuteWithDeadline(hook *Hook, event *Event,
    deadline time.Time) *ExecutionResult
```

Executor Query Methods

```
func (e *Executor) GetResults(n int) []*ExecutionResult
func (e *Executor) GetAllResults() []*ExecutionResult
func (e *Executor) GetResultsByStatus(status HookStatus)
    []*ExecutionResult
func (e *Executor) GetStatistics() *ExecutorStatistics
```

Executor Control Methods

```
func (e *Executor) ClearResults()
func (e *Executor) SetMaxConcurrent(max int)
func (e *Executor) SetMaxResults(max int)
```

Executor Callback Methods

```
func (e *Executor) OnComplete(callback ResultCallback)
func (e *Executor) OnError(callback ResultCallback)
```

ExecutionResult Methods

```
func NewExecutionResult(hook *Hook) *ExecutionResult
func (r *ExecutionResult) Complete(err error)
func (r *ExecutionResult) Cancel()
func (r *ExecutionResult) Skip()
func (r *ExecutionResult) String() string
```

Integration Patterns

Integration with Task System

```
type TaskManager struct {
    hooks *hooks.Manager
}

func (tm *TaskManager) ExecuteTask(task *Task) error {
    // Trigger before hook
    beforeEvent := hooks.NewEvent(hooks.HookTypeBeforeTask)
    beforeEvent.SetData("task_id", task.ID)
```



```

beforeEvent.SetData("task_type", task.Type)
beforeEvent.SetMetadata("user", task.Owner)

results := tm.hooks.TriggerEventAndWait(beforeEvent)

// Check for validation failures
for _, result := range results {
    if result.Status == hooks.StatusFailed {
        return fmt.Errorf("hook validation failed: %w",
            result.Error)
    }
}

// Execute task
start := time.Now()
err := task.Execute()
duration := time.Since(start)

// Trigger after hook
afterEvent := hooks.NewEvent(hooks.HookTypeAfterTask)
afterEvent.SetData("task_id", task.ID)
afterEvent.SetData("duration", duration)
afterEvent.SetData("success", err == nil)
if err != nil {
    afterEvent.SetData("error", err)
}

tm.hooks.TriggerEvent(afterEvent) // Async

return err
}

```

Integration with LLM Provider

```

type LLMPProvider struct {
    hooks *hooks.Manager
}

func (lp *LLMPProvider) Generate(ctx context.Context, prompt
    string, maxTokens int) (string, error) {
    // Before LLM hook
    beforeEvent := hooks.NewEventWithContext(ctx,
        hooks.HookTypeBeforeLLM)
    beforeEvent.SetData("prompt", prompt)
    beforeEvent.SetData("max_tokens", maxTokens)

```

```

beforeEvent.SetData("model", lp.modelName)

results := lp.hooks.TriggerEventAndWait(beforeEvent)

// Check for cached response
for _, result := range results {
    if cached, ok := result.HookName == "cache-check"; ok {
        if response, found :=
            beforeEvent.GetData("cached_response"); found {
            return response.(string), nil
        }
    }
}

// Make LLM call
start := time.Now()
response, err := lp.callLLM(ctx, prompt, maxTokens)
duration := time.Since(start)

// After LLM hook
afterEvent := hooks.NewEventWithContext(ctx,
    hooks.HookTypeAfterLLM)
afterEvent.SetData("prompt", prompt)
afterEvent.SetData("response", response)
afterEvent.SetData("duration", duration)
afterEvent.SetData("tokens_used", len(response)/4) // Rough
estimate
if err != nil {
    afterEvent.SetData("error", err)
}

lp.hooks.TriggerEvent(afterEvent) // Async

return response, err
}

```

Integration with Build System

```

type BuildSystem struct {
    hooks *hooks.Manager
}

func (bs *BuildSystem) Build(ctx context.Context, project
    string) error {
    // Before build hook

```

```

beforeEvent := hooks.NewEventWithContext(ctx,
    hooks.HookTypeBeforeBuild)
beforeEvent.SetData("project", project)
beforeEvent.SetData("target", "production")

results := bs.hooks.TriggerEventAndWait(beforeEvent)

// Check for validation errors
for _, result := range results {
    if result.Status == hooks.StatusFailed {
        return fmt.Errorf("pre-build check failed: %w",
            result.Error)
    }
}

// Build
start := time.Now()
output, err := bs.runBuild(ctx, project)
duration := time.Since(start)

// After build hook
afterEvent := hooks.NewEventWithContext(ctx,
    hooks.HookTypeAfterBuild)
afterEvent.SetData("project", project)
afterEvent.SetData("duration", duration)
afterEvent.SetData("success", err == nil)
afterEvent.SetData("output", output)
if err != nil {
    afterEvent.SetData("error", err)
}

bs.hooks.TriggerEvent(afterEvent)

return err
}

```

Integration with Test Runner

```

type TestRunner struct {
    hooks *hooks.Manager
}

func (tr *TestRunner) RunTests(ctx context.Context)
    (*TestResults, error) {
    // Before test hook

```

```

beforeEvent := hooks.NewEventWithContext(ctx,
    hooks.HookTypeBeforeTest)
beforeEvent.SetData("test_suite", "all")

tr.hooks.TriggerEventAndWait(beforeEvent)

// Run tests
start := time.Now()
results, err := tr.executeTests(ctx)
duration := time.Since(start)

// After test hook
afterEvent := hooks.NewEventWithContext(ctx,
    hooks.HookTypeAfterTest)
afterEvent.SetData("passed", results.Passed)
afterEvent.SetData("failed", results.Failed)
afterEvent.SetData("skipped", results.Skipped)
afterEvent.SetData("duration", duration)
if err != nil {
    afterEvent.SetData("error", err)
}

tr.hooks.TriggerEvent(afterEvent)

return results, err
}

```

Plugin System

```

type PluginManager struct {
    hooks *hooks.Manager
}

func (pm *PluginManager) LoadPlugin(plugin Plugin) error {
    // Let plugin register hooks
    plugin.RegisterHooks(pm.hooks)
    return nil
}

// Example plugin
type LoggingPlugin struct{}

func (lp *LoggingPlugin) RegisterHooks(manager *hooks.Manager) {
    // Register multiple hooks

```

```

manager.RegisterMany([]*hooks.Hook{
    hooks.NewHook("plugin-log-task",
        hooks.HookTypeBeforeTask,
        func(ctx context.Context, event *hooks.Event) error {
            taskID, _ := event.GetData("task_id")
            log.Printf("[PLUGIN] Task: %v\n", taskID)
            return nil
        },
    ),
    hooks.NewHook("plugin-log-llm", hooks.HookTypeBeforeLLM,
        func(ctx context.Context, event *hooks.Event) error {
            prompt, _ := event.GetData("prompt")
            log.Printf("[PLUGIN] LLM Call: %s\n", prompt)
            return nil
        },
    ),
})
}

```

FAQ

Q: What's the difference between sync and async hooks?

A: Synchronous hooks block the caller until they complete, while asynchronous hooks execute in separate goroutines and don't block. Use sync hooks when the operation must complete before continuing (e.g., validation), and async hooks for independent operations (e.g., logging, notifications).

Q: How many hooks can I register?

A: There's no hard limit, but performance degrades with many hooks per event. Keep it under 50 hooks per event type for best performance.

Q: Can hooks modify event data?

A: Yes! Hooks can read and write event data using `SetData()`. However, be careful with async hooks as they may race with each other.

Q: What happens if a hook fails?

A: The hook's result will have `Status = StatusFailed` and the error will be in `result.Error`. Other hooks will still execute (unless context is canceled). For sync hooks before an operation, you can check results and abort if validation fails.

Q: Can I cancel hook execution?

A: Yes, by canceling the context:

```
ctx, cancel := context.WithTimeout(context.Background(),
    5*time.Second)
defer cancel()

results := manager.Trigger(ctx, hooks.HookTypeBeforeTask)
```

Q: How do I debug hooks?

A: Use lifecycle callbacks:

```
manager.OnExecute(func(event *hooks.Event, results
    []*hooks.ExecutionResult) {
    for _, result := range results {
        log.Printf("%s: %s\n", result.HookName, result.Status)
        if result.Error != nil {
            log.Printf("    Error: %v\n", result.Error)
        }
    }
})
```

Q: Can I have multiple managers?

A: Yes! Each manager is independent and thread-safe. This is useful for isolating different subsystems.

Q: What's the overhead of the hooks system?

A: Minimal for async hooks (~0.01ms per hook). Sync hooks have the overhead of the hook function itself. The system uses efficient priority sorting and minimal locking.

Q: Can I dynamically enable/disable hooks?

A: Yes! Use `manager.Enable(id)` and `manager.Disable(id)`. Disabled hooks are skipped during execution.

Q: How do I handle hook errors?

A: Check execution results:

```
results := manager.TriggerEvent(event)
for _, result := range results {
```

```

    if result.Status == hooks.StatusFailed {
        log.Printf("Hook %s failed: %v\n", result.HookName,
            result.Error)
    }
}

```

Or use executor callbacks:

```

executor.OnError(func(result *hooks.ExecutionResult) {
    notifier.SendAlert(result.Error)
})

```

Troubleshooting

Hook Not Executing

Symptoms: Hook registered but doesn't execute when event is triggered.

Possible Causes: 1. Hook is disabled 2. Event type doesn't match hook type 3. Condition returns false 4. Hook not enabled

Solutions:

```

// Check if hook is enabled
hook, _ := manager.Get(hookID)
if !hook.Enabled {
    manager.Enable(hookID)
}

// Check hook type
if hook.Type != event.Type {
    log.Printf("Type mismatch: hook=%s, event=%s\n", hook.Type,
        event.Type)
}

// Check condition
if hook.Condition != nil && !hook.Condition(event) {
    log.Printf("Condition failed for hook %s\n", hook.Name)
}

```

Hook Timeout

Symptoms: Hook times out even though it should complete quickly.

Cause: Handler doesn't respect context cancellation.

Solution: Always check `ctx.Done()`:

```
func(ctx context.Context, event *hooks.Event) error {
    select {
    case result := <-doWork():
        return result
    case <-ctx.Done():
        return ctx.Err() // Respect timeout
    }
}
```

Async Hooks Not Completing

Symptoms: Async hooks start but never complete.

Cause: Not waiting for async hooks.

Solution:

```
// Option 1: Trigger and wait
results := manager.TriggerAndWait(ctx, hooks.HookTypeBeforeTask)

// Option 2: Wait later
results := manager.TriggerEvent(event)
// ... do other work ...
manager.Wait() // Wait for all async hooks
```

High Memory Usage

Symptoms: Memory grows over time.

Cause: Executor storing too many results.

Solution: Limit stored results:

```
executor := manager.GetExecutor()
executor.SetMaxResults(100) // Keep only 100 most recent
```

Slow Hook Execution

Symptoms: Hook trigger takes a long time.

Possible Causes: 1. Too many sync hooks 2. Hooks not using async appropriately
3. Hooks doing heavy computation

Solutions:


```
// 1. Make independent hooks async
hook.Async = true

// 2. Increase concurrency limit
executor.SetMaxConcurrent(20)

// 3. Profile hooks
stats := executor.GetStatistics()
fmt.Printf("Avg Duration: %v\n", stats.AverageDuration)

// 4. Check individual results
results := executor.GetAllResults()
for _, result := range results {
    if result.Duration > 100*time.Millisecond {
        log.Printf("Slow hook: %s (%v)\n", result.HookName,
            result.Duration)
    }
}
```

Duplicate Hook ID Error

Symptoms: Register() returns “hook with ID already registered”.

Cause: Hook ID collision (rare but possible).

Solution: IDs are auto-generated with timestamp. If registering many hooks rapidly, add small delay:

```
for _, hook := range hooks {
    manager.Register(hook)
    time.Sleep(1 * time.Microsecond) // Ensure unique timestamps
}
```

Race Conditions

Symptoms: Inconsistent behavior with concurrent access.

Cause: Improper concurrent access to shared data in hooks.

Solution: The manager itself is thread-safe, but your hook handlers must handle their own synchronization:

```
var mu sync.Mutex
var counter int

func(ctx context.Context, event *hooks.Event) error {
```

```
    mu.Lock()
    counter++
    mu.Unlock()
    return nil
}
```

Hook Validation Errors

Symptoms: Register() returns validation error.

Common Errors: - Empty hook ID - Empty hook name - Empty hook type - Nil handler - Invalid priority (< 1 or > 100)

Solution:

```
hook := hooks.NewHook("name", hooks.HookTypeBeforeTask, handler)
// All required fields are set
// Priority defaults to PriorityNormal

if err := hook.Validate(); err != nil {
    log.Printf("Invalid hook: %v\n", err)
}
```

Summary

The Hooks System provides a powerful, flexible event-driven architecture for HelixCode:

- **13 hook types** covering all lifecycle events
- **5 priority levels** for execution control
- **Sync and async** execution modes
- **Thread-safe** manager and executor
- **Rich event system** with data, metadata, and context
- **Comprehensive statistics** for monitoring
- **Lifecycle callbacks** for event handling
- **Conditional execution** for targeted hooks
- **Timeout support** per hook
- **Tag-based organization** for categorization

Use hooks for logging, metrics, validation, notifications, auditing, caching, and more. The system is production-ready with 52.6% test coverage and full concurrency support.

Document Version: 1.0 **Package Version:** dev.helix.code/internal/hooks **Go Version:** 1.24.0 **Last Updated:** November 7, 2025

Next Steps: 1. Explore [Common Use Cases](#) 2. Review [Best Practices](#) 3. Integrate with your application 4. Monitor with statistics

End of Hooks System User Guide